

Reading Notes: GeCCo

Guided Generation of Computational Cognitive Models

An Annotated Summary

Contents

1	Abstract & Overview	2
2	Motivation: The Bottleneck in Cognitive Modelling	2
3	LLMs as a Solution	2
4	The GeCCo Pipeline	3
4.1	Structure of Each Iteration	3
4.2	Fitting and Evaluation	3
4.3	Data Splits	3
4.4	Fail-safes	3
5	Metrics	3
6	Evaluation: Four Cognitive Domains	4
6.1	Experiment 1: Decision Making	4
6.2	Experiment 2: Learning	4
6.3	Experiment 3: Planning	4
6.4	Experiment 4: Working Memory	5
7	Control Experiments	5
7.1	Contribution of LLM Features	5
7.2	Prompt Ablation Study	5
7.3	Data Contamination	5
7.4	Ground Truth Recovery	5
7.5	Comparison with CENTAUR	6
8	Discussion	6
8.1	Main Interpretation	6
8.2	Strengths of the Approach	6
8.3	Limitations	7
8.4	Future Directions	7
8.5	Conclusion	7

Abstract & Overview

The GeCCo pipeline uses Large Language Models (LLMs) to automatically generate, fit, and iteratively refine computational cognitive models as Python functions. Given a task description, participant behavioural data, and a template function, GeCCo prompts an LLM to propose candidate models, fits those proposals to held-out data, and refines them based on feedback derived from predictive performance.

The approach was benchmarked across four cognitive domains—**decision making**, **learning**, **planning**, and **memory**—using three open-source LLMs of varying size, capacity, and training family. On all four human behavioural datasets, LLM-generated models consistently matched or outperformed the best domain-specific models from the literature, while remaining interpretable.

Code: <https://github.com/MilenaCCNlab/gecco.git>

Motivation: The Bottleneck in Cognitive Modelling

Computational cognitive models provide a formal framework for understanding human behaviour. In practice, researchers hand-craft these models, fit them to behavioural data, and iteratively refine them—a process that is:

- **Slow.** Translating a psychological theory into a working algorithm requires lengthy literature reviews and repeated trial-and-error coding cycles.
- **Skill-intensive.** Researchers must simultaneously be domain experts, theoreticians, and software engineers—a rare combination.
- **Biased by theoretical commitments.** Researchers tend to explore only the model classes they already know and favour, potentially missing superior explanations for the data.

This “streetlight effect” artificially shrinks the space of models considered, motivating an automated approach that is not anchored to any particular theoretical school.

LLMs as a Solution

Three specific LLM capabilities make them well-suited to cognitive model generation:

1. **Natural language flexibility.** LLMs can process task descriptions and behavioural data written in plain English, adapting across diverse experimental paradigms without a custom interface for each.
2. **In-context pattern recognition.** Drawing on broad pre-training knowledge, LLMs can identify structure in complex behavioural data and generate plausible psychological hypotheses.
3. **Code synthesis.** LLMs can translate hypotheses directly into executable Python functions, closing the loop from theory to testable model instantly.

These three abilities have already shown promise in automated statistical modelling (probabilistic program generation), and GeCCo applies the same logic to cognitive science.

The GeCCo Pipeline

Structure of Each Iteration

Each pipeline run consists of **10 sampling iterations**, repeated across **5 independent runs** per domain. On every iteration the LLM receives a structured prompt containing:

1. A natural language **task description**.
2. **Behavioural data** from a subset of participants (the *prompt* split).
3. **Instructions and guardrails** specifying the required Python function signature.
4. A **domain-specific template model** for syntactic guidance.
5. **Iterative feedback** (from iteration 2 onward): the current best-performing model and a list of previously used parameter names.

The LLM generates **three distinct cognitive models** with non-overlapping parameter sets.

Fitting and Evaluation

Generated models are parsed and converted to executable Python via `exec()`. Each model is then fit to a held-out *validation* dataset using `scipy.optimize.minimize` initialised from 20 random starting points. Model quality is scored with the **Bayesian Information Criterion (BIC)**—lower is better. The best-performing model is fed back into the next prompt.

Data Splits

Three separate data partitions prevent data leakage:

Split	Seen by LLM?	Purpose
Prompt data	Yes	Provides context for hypothesis generation
Validation data	No	Scores candidate models during the loop
Test data	No	Final evaluation against human baselines

Fail-safes

If a generated model contains syntax or runtime errors, the iteration restarts automatically—no human intervention required.

Metrics

- **BIC (Bayesian Information Criterion)**. Balances goodness of fit against model complexity by penalising the number of free parameters. When two models fit equally well, BIC favours the simpler one. The pipeline is metric-agnostic; AIC results are reported in Appendix I.
- **Exceedance Probability (EXP)**. Used in the final evaluation stage. Quantifies the posterior probability that a given model provides the most prevalent explanation

of observed behaviour across the population, translating raw BIC scores into an interpretable confidence measure.

Evaluation: Four Cognitive Domains

Four canonical paradigms were selected to form an *escalating complexity* ladder—from static decisions to rich interactions between memory systems.

Experiment 1: Decision Making

Task. Participants chose between two options defined by four features, each with a different validity (importance).

Baseline. The probabilistic Weighted Additive Decision (pWADD) model, which uses four separate parameters to weight the four features.

Result. The Llama-generated model achieved a substantially lower BIC (39.40 vs. 51.97; $p < .001$).

Insight. The LLM reduced four parameters to one *discount factor* that smoothly interpolates between three classic decision strategies—*Take the Best*, *Equal Weighting*, and *Weighted Additive*—mirroring complex Bayesian inference but arriving at the solution naturally.

Experiment 2: Learning

Task. A two-armed bandit with full feedback: after each trial participants also saw what the unchosen option would have paid, introducing regret and relief.

Baseline. A four-learning-rate variant of the Rescorla–Wagner model ($RW_{4\alpha}$), plus a “choice stickiness” (perseveration) parameter.

Result. BIC of 77.97 vs. 85.24; EXP = 0.97.

Insight. The LLM replaced the asymmetric learning rates with a clean separation between actual and counterfactual value traces (*fictive trace*), and replaced the perseveration hack with a psychologically motivated *forgetting parameter* that decays unchosen-option values over time—mirroring the limits of working memory.

Experiment 3: Planning

Task. The classic two-stage Daw task, in which choices lead to a second stage probabilistically, requiring forward planning.

Baseline. A Hybrid model combining model-free (SARSA) and model-based value iteration, averaged by a weighting parameter.

Result. BIC of 432.47 vs. 437.38; EXP = 0.85. (The BIC difference did not reach conventional significance at $p = .27$, but the population-level advantage is clear in EXP.)

Insight. The LLM unified the model-free/model-based dichotomy into a single system using two transition-dependent learning rates, and omitted the temporal discounting parameter, finding it unnecessary—aligning with recent theoretical calls to abandon strict dual-system accounts of planning.

Experiment 4: Working Memory

Task. Trial-and-error stimulus–response learning under varying cognitive load (3 vs. 6 associations held in mind simultaneously).

Baseline. The RL-WM model (Collins & Frank, 2012), in which a slow RL module and a fast but capacity-limited WM module operate independently, with a load-dependent mixing weight.

Result. BIC of 267.25 vs. 275.20; EXP = 0.99.

Insight. Contrary to the standard assumption that RL is immune to cognitive load, the LLM’s model assigned separate RL learning rates for low- and high-load conditions (`lr_low` / `lr_high`), discovering that load modulates the core learning signal—a conclusion consistent with neuroimaging evidence on reward prediction error signals.

Control Experiments

Contribution of LLM Features

A mixed-effects regression predicting BIC from *reasoning ability*, *model family*, and *model size* found that:

- Llama outperformed Qwen ($\beta = 5.624$, $p = .03$) despite Qwen having more parameters, suggesting training data and protocol matter more than raw scale.
- Reasoning ability showed a marginal advantage ($\beta = -0.266$, $p = .07$).

Prompt Ablation Study

Systematically removing each prompt component and refitting a mixed-effects model on post-ablation BIC scores revealed the following ranked importance:

Component	β	p-value
Iterative feedback	−17.040	< .05
Participant data	−12.836	< .05
Task description	−9.834	< .05
Code template	−9.359	< .05

Iterative feedback is the single most important driver of performance.

Data Contamination

The LogProber method was applied to Llama prompts across all four domains. Acceleration scores were well below the contamination threshold of 1.0 (range: 0.017–0.648), confirming that the LLM is reasoning about novel problems rather than recalling memorised solutions.

Ground Truth Recovery

On synthetic datasets with known generating processes, the LLM successfully reverse-engineered the true model in both learning (Rescorla–Wagner) and decision-making (Take the Best; Tallying) domains, validating the mathematical soundness of the approach.

Comparison with CENTAUR

CENTAUR is a large foundation model trained to predict human behaviour across cognitive tasks, serving as a proxy for the ceiling of explainable variance. GeCCo matched or exceeded CENTAUR’s negative log-likelihood across all four domains, while producing interpretable Python code rather than opaque model weights.

Discussion

Overall, GeCCo shows that large language models can be used as practical tools for cognitive model discovery. The main contribution of the paper is not simply that an LLM can write Python code. More importantly, the LLM-generated models are fitted to held-out human behavioural data and are compared with established models from the cognitive science literature. This makes the pipeline more scientific than a one-shot prompt: the LLM proposes candidate mechanisms, but the data decide which model is kept.

Main Interpretation

The strongest result is that the LLM-generated models matched or outperformed the best literature models across four different cognitive domains: decision making, learning, planning, and working memory. This suggests that LLMs can help researchers explore a wider model space than they might normally consider by hand. In several cases, GeCCo did not only reproduce known ideas, but also proposed simpler or alternative explanations. For example, in the decision-making task, the LLM-generated model used one discount factor to move between classic heuristics such as Take-the-Best, Equal Weighting, and Weighted Additive choice. In the learning task, the model separated experienced outcomes from counterfactual outcomes, which gives a clearer interpretation of regret or relief. In the working-memory task, the model suggested that cognitive load may influence the reinforcement-learning rate itself, rather than only changing the weight of a separate working-memory system.

This is important because cognitive modelling is often limited by the assumptions researchers start with. If a researcher only tests familiar models, they may miss a simpler or more plausible explanation. GeCCo can therefore be understood as a hypothesis-generation tool: it expands the set of possible models, but the final interpretation still requires human scientific judgement.

Strengths of the Approach

A major strength of GeCCo is its combination of automation and interpretability. Unlike black-box prediction models, the output of GeCCo is readable Python code. Researchers can inspect the model, understand its assumptions, and compare it with existing theories. This makes the approach more useful for cognitive science than a purely predictive model that gives good accuracy but no explanation.

Another strength is the feedback loop. The ablation experiments show that iterative feedback is the most important prompt component. This means that GeCCo is not successful just because the LLM produces a good model immediately. Instead, the pipeline

improves through repeated cycles of generation, model fitting, BIC evaluation, and feedback. This is close to how human researchers revise models, but it can happen much faster.

The use of held-out validation and test data is also important. The LLM sees only the prompt data, while the generated models are evaluated on separate data. This reduces the risk that the model simply overfits the examples in the prompt. The additional contamination analysis and synthetic recovery experiments further support the claim that the pipeline is not merely memorising known results.

Limitations

Despite these strengths, GeCCo should not be treated as a replacement for cognitive scientists. First, the quality of the generated models still depends on the prompt design, the data representation, and the template. Even if the template had the weakest effect in the ablation study, it may still guide the LLM toward certain model families and away from others. This means that the pipeline is not completely theory-free.

Second, the feedback signal is mainly based on predictive performance, especially BIC. A model with a lower BIC is not automatically the best scientific explanation. It may fit the data well but still be difficult to justify theoretically. For this reason, human researchers still need to evaluate whether the mechanisms are psychologically meaningful, biologically plausible, and consistent with prior literature.

Third, the paper focuses on controlled laboratory tasks with relatively structured behavioural data. It remains unclear how well GeCCo would work on more complex cognitive domains such as language, perception, social interaction, or naturalistic behaviour. These domains may require richer data formats, more parameters, and more complicated model structures.

Finally, there is a risk of over-trusting LLM-generated explanations. Because the generated code can look clean and convincing, researchers may accept it too quickly. The model should therefore be treated as a candidate hypothesis, not as a final theory.

Future Directions

Future work could improve GeCCo in several ways. One direction is to add richer feedback than BIC alone. For example, a second LLM or a human expert could provide feedback on theoretical plausibility, parameter interpretability, and whether the model violates known cognitive constraints. Another direction is to test the pipeline on broader datasets and more naturalistic tasks, where human behaviour is less controlled and more noisy.

It would also be useful to study whether GeCCo can generate models that generalise across tasks. A strong cognitive theory should not only fit one dataset, but also explain behaviour in related tasks. Cross-task generalisation would therefore be an important next step for evaluating whether LLM-generated models are scientifically robust.

Conclusion

In conclusion, GeCCo demonstrates a promising new role for LLMs in cognitive science. The LLM is not used as a final authority, but as a fast generator of executable model hypotheses. The scientific value comes from combining this generative ability with traditional model fitting, held-out evaluation, and human interpretation. Therefore, the most reasonable

view is that GeCCo can accelerate cognitive modelling by helping researchers discover, compare, and refine candidate explanations of human behaviour more efficiently.

Reference

Milena et al. (2024). *GeCCo: Guided Generation of Computational Cognitive Models*. Available at: <https://github.com/MilenaCCNlab/gecco.git>